12. Integrated Swift Programming

Task 1:

Code:

```
import Foundation
protocol Borrowable {
    var id: Int { get }
    var title: String { get }
    var isAvailable: Bool { get set }
}
struct Book: Borrowable {
    let id: Int
    let title: String
    var isAvailable: Bool = true
    let author: String
}
struct Magazine: Borrowable {
    let id: Int
    let title: String
    var isAvailable: Bool = true
    let issue: Int
enum LibraryError: Error {
    case itemNotFound(id: Int)
    case alreadyBorrowed(title: String)
    case limitReached(max: Int)
}
final class Member {
    let name: String
```

```
    private(set) var borrowedIDs: [Int] = []
    static let limit = 2
    init(name: String) {
        self.name = name
    }
    func recordBorrow(_ id: Int) throws {
        guard borrowedIDs.count < Self.limit else {
            throw LibraryError.limitReached(max: Self.limit)
        }
        borrowedIDs.append(id)
    }
    func recordReturn(_ id: Int) {
        borrowedIDs.removeAll { $0 == id }
    }
}
struct Catalog<Item: Borrowable> {
    private var items: [Int: Item] = [:]
    mutating func add(_ item: Item) {
        items[item.id] = item
    }
    mutating func checkout(id: Int, to member: Member) throws {
        guard var item = items[id] else {
            throw LibraryError.itemNotFound(id: id)
        }
        guard item.isAvailable else {
            throw LibraryError.alreadyBorrowed(title: item.title)
        }
```

```
        try member.recordBorrow(id)
        item.isAvailable = false
        items[id] = item
        print("\(item.title) borrowed by \(member.name)")
    }
}
var library = Catalog<Book>()
library.add(Book(id: 1, title: "Swift Basics",
                 author: "John"))
library.add(Book(id: 2, title: "iOS Development",
                 author: "Sam"))
library.add(Book(id: 3, title: "Programming",
                 author: "Alex"))
let member = Member(name: "Brinda")
do {
    try library.checkout(id: 1, to: member)
    try library.checkout(id: 1, to: member)
} catch LibraryError.itemNotFound(let id) {
    print("Item \(id) not found")
} catch LibraryError.alreadyBorrowed(let title) {
    print("\(title) is already borrowed")
} catch LibraryError.limitReached(let max) {
    print("Borrowing limit reached: \(max)")
} catch {
    print("Unknown error")
}
do {
```

```
    try library.checkout(id: 2, to: member)
    try library.checkout(id: 3, to: member)
} catch LibraryError.itemNotFound(let id) {
    print("Item \(id) not found")
} catch LibraryError.alreadyBorrowed(let title) {
    print("\(title) is already borrowed")
} catch LibraryError.limitReached(let max) {
    print("Borrowing limit reached: \(max)")
} catch {
    print("Unknown error")
}
```

Output:

Output

```
Swift Basics borrowed by Brinda
Swift Basics is already borrowed
iOS Development borrowed by Brinda
Borrowing limit reached: 2
```

Task 2:

Code:

```
import Foundation
protocol Entity {
    var id: Int { get }
}
enum RepositoryError: Error {
    case duplicateID(Int)
```

```
    case notFound(Int)
}
struct Repository<T: Entity> {
    private var storage: [Int: T] = [:]
    mutating func add(_ item: T) throws {
        guard storage[item.id] == nil else {
            throw RepositoryError.duplicateID(item.id)
        }
        storage[item.id] = item
    }
    func find(_ id: Int) -> T? {
        return storage[id]
    }
    mutating func remove(_ id: Int) throws {
        guard storage.removeValue(forKey: id) != nil else {
            throw RepositoryError.notFound(id)
        }
    }
    func all(where predicate: (T) -> Bool) -> [T] {
        return storage.values
            .filter(predicate)
            .sorted { $0.id < $1.id }
    }
}
struct Student: Entity {
    let id: Int
    let name: String
```

```
    let marks: Int
}
struct Course: Entity {
    let id: Int
    let title: String
}
var students = Repository<Student>()
do {
    try students.add(Student(id: 2, name: "Brinda", marks: 85))
    try  students.add(Student(id:  1,  name:  "Deepika",  marks:
    92))
    try  students.add(Student(id:  1,  name:  "Ranisha",  marks:
    75))
} catch RepositoryError.duplicateID(let id) {
    print("Duplicate student ID: \(id)")
} catch {
    print("Repository error")
}
print("Students scoring above 80:")
for student in students.all(where: { $0.marks > 80 }) {
    print("\(student.id): \(student.name) - \(student.marks)")
}
print("Find student:", students.find(2)?.name ?? "Not found")
do {
    try students.remove(2)
    print("Student removed")
    try students.remove(5)
} catch RepositoryError.notFound(let id) {
    print("Record \(id) not found")
```

```
} catch {
    print("Repository error")
}
var courses = Repository<Course>()
do {
    try courses.add(Course(id: 101, title: "Swift"))
    try courses.add(Course(id: 102, title: "Java"))
} catch {
    print("Course error")
}
for course in courses.all(where: { $0.id >= 101 }) {
    print("Course: \(course.id) - \(course.title)")
}
```

Output:

Output

```
Duplicate student ID: 1
Students scoring above 80:
1: Deepika - 92
2: Brinda - 85
Find student: Brinda
Student removed
Record 5 not found
Course: 101 - Swift
Course: 102 - Java
```

Task 3:

Code:

```
import Foundation
protocol Entity {
```

```
    var id: Int { get }
}
protocol Describable {
    var summary: String { get }
}
enum RepositoryError: Error {
    case duplicateID(Int)
    case notFound(Int)
}
struct Repository<T: Entity> {
    private var storage: [Int: T] = [:]
    mutating func add(_ item: T) throws {
        guard storage[item.id] == nil else {
            throw RepositoryError.duplicateID(item.id)
        }
        storage[item.id] = item
    }
    func find(_ id: Int) -> T? {
        return storage[id]
    }
    mutating func remove(_ id: Int) throws {
        guard storage.removeValue(forKey: id) != nil else {
            throw RepositoryError.notFound(id)
        }
    }
    func all(where predicate: (T) -> Bool) -> [T] {
        storage.values
```

```
            .filter(predicate)
            .sorted { $0.id < $1.id }
    }
}
struct Student: Entity, Describable {
    let id: Int
    let name: String
    let marks: Int

    var summary: String {
        return "Student \(id): \(name), Marks: \(marks)"
    }
}
struct Course: Entity, Describable {
    let id: Int
    let title: String

    var summary: String {
        return "Course \(id): \(title)"
    }
}
extension Repository where T: Describable {
    func printReport() {
        print("Repository Report")
        for item in all(where: { _ in true }) {
            print(item.summary)
        }
```

```
    }
}
var studentRepo = Repository<Student>()
try studentRepo.add(Student(id: 1, name: "Brinda", marks: 90))
try studentRepo.add(Student(id: 2, name: "Deepika", marks: 85))

 studentRepo.printReport()
var courseRepo = Repository<Course>()
try courseRepo.add(Course(id: 101, title: "Swift"))
try courseRepo.add(Course(id: 102, title: "Java"))
courseRepo.printReport()
struct Department: Entity {
    let id: Int
}
```

Output:

Output

```
Repository Report
Student 1: Brinda, Marks: 90
Student 2: Deepika, Marks: 85
Repository Report
Course 101: Swift
Course 102: Java
```